HES-SO VALAIS-WALLIS • TRAVAUX PRATIQUES D'INSTRUMENTATION

Dossier de Résolution, Synthèse Théorique et Codes Sources

Corrigé méthodologique complet : (1) Codeur optique incrémental HEDL-5540, (2) Station météo Raspberry Pi Pico 2 BME280/OLED, (3) Thermomètre à résistance de platine Pt100

Vue d'ensemble des 3 laboratoires : Ce document rassemble pour chacun des trois travaux pratiques un résumé détaillé de ce qui se passe physiquement, la méthode de résolution pas à pas, les réponses démontrées à toutes les questions théoriques et pratiques, ainsi que l'intégralité des codes sources complets (Python, MicroPython, MATLAB et LabVIEW) prêts à être copiés.

Laboratoire	Capteur / Système étudié	Phénomènes physiques & Métrologie	Tous les fichiers de code inclus dans ce PDF
TP 1 : Codeur optique	Codeur incrémental optique HEDL-5540 (500 stries/tr) sur moteur DC	Signaux en quadrature (A, B) et index (I), mesure de vitesse par comptage d'impulsions (T_E) vs durée de période (f_{clk}), bruit de quantification	• Commandes terminal & Série • `labo1_section3_statique.py` • `labo1_section4_dynamique.py` (complet + méthode optimale hybride)
TP 2 : Station météo Pico 2	Raspberry Pi Pico 2 (RP2350), BME280/BMP280, OLED SSD1306 (I2C)	Bus I2C partagé, pression barométrique vs altitude, réduction du bruit aléatoire par moyennage ($\sigma / \sqrt{N}$), temps de conversion	• `commandes_repl_exploratoires.py` • `etape1_clignote.py` • `etape2_bme280_fonction.py` • `etape3_oled_fonction.py` • `meteo_station_sans_filtre.py` • `meteo_station.py` (ou `main.py`) • `etape5_bruit_pression.py`
TP 3 : Sonde au platine Pt100	Sonde Pt100 Jumo classe B (2,5 m), référence Testo 720, bain agité	Loi IEC 60751 (Callendar-Van Dusen), résistance des fils (2 fils), auto-échauffement par effet Joule, calibration et temps de réponse ($t_{0,5}$, $t_{0,9}$)	• Formules LabVIEW (Formula Node) • `labo3_complet_python.py` (Sections 4.1 à 4.6) • `labo3_complet_matlab.m` (Sections 4.1 à 4.6)

LABO 1 : Codeur incrémental optique (HEDL-5540) et Mesure de vitesse

1.1 Résumé de ce qui se passe et comment résoudre le TP étape par étape

Dans ce premier laboratoire, on étudie comment mesurer la position angulaire et la vitesse de rotation d'un moteur à courant continu à l'aide d'un **codeur optique incrémental HEDL-5540** possédant un disque de **N = 500 stries par tour**. Voici le déroulement concret du TP et comment le résoudre :

1. **Analyse matérielle et oscilloscope (Sections 2.1 à 2.3) :** Une LED infrarouge éclaire le disque strié en rotation devant des photodétecteurs. Le capteur sort trois signaux logiques TTL (0–5 V) : **A** et **B** (deux signaux carrés décalés d'un quart de période, soit 90° électriques, ce qui permet de connaître le sens de rotation et de multiplier la résolution par 4) et **I** (une impulsion unique par tour servant de top zéro). À l'oscilloscope, on vérifie l'inversion du déphasage entre +U et −U, puis on déclenche (trigger) sur le signal I pour vérifier que la largeur de I vaut un quart de période de A et qu'il y a bien 500 périodes de A entre deux impulsions de I.
2. **Prise en main du boîtier d'acquisition USB (Section 2.4) :** Le boîtier noir contient un microcontrôleur/FPGA qui compte les flancs du codeur et communique avec le PC via un port série virtuel (COM). Le script `comp_encodeur_texte.py` affiche en temps réel : (1) la position en incréments, (2) le nombre d'incréments par milliseconde $\Delta P = P[k] - P[k-1]$, et (3) le nombre de coups d'horloge à 1 MHz entre deux fronts montants de A.
3. **Comparaison des 2 méthodes à vitesse constante (Section 3) :** On règle une tension constante (ex. 5 V), on mesure la vitesse réelle du moteur avec le tachymètre optique **Testo 460**, puis on compare deux méthodes de dérivation discrète $\omega = \Delta\theta / \Delta t$:
   - **Méthode 1 (Section 3.1 - Comptage d'impulsions sur T_E = 1 ms) :** On fixe $\Delta t = T_E = 1$ ms et on compte ΔP. La relation est linéaire ($\Delta P \propto$ vitesse), idéale à haute vitesse, mais aveugle ou très imprécise à basse vitesse (erreur constante de 2 tr/s = 120 tr/min).
   - **Méthode 2 (Section 3.2 - Mesure de période avec f_{clk} = 1 MHz) :** On fixe $\Delta\theta$ = 1 strie (1/500 tour) et on mesure la durée Δt en comptant les impulsions d'une horloge à 1 MHz (`period`). La relation est hyperbolique (`period` $\propto$ 1/vitesse), ultra précise à très basse vitesse, mais très bruitée à haute vitesse car `period` devient petit.
4. **Analyse dynamique lors d'une accélération (Section 4) :** On enregistre un démarrage complet du moteur avec `comp_encodeur_V1.py` dans un fichier CSV/NPY, puis on écrit un script Python pour tracer les deux vitesses sur le même graphe, observer le bruit de quantification et calculer la vitesse exacte de croisement où les deux méthodes ont la même résolution.

1.2 Section 2.1 : Caractéristiques du codeur HEDL-5540 (Datasheet & Banc)

- **Nombre de stries du capteur :** D'après la plaque signalétique du banc d'essai (*Encoder data : Counts per turn = 500*) et la fiche technique du HEDL-5540 standard, le disque optique comporte **N = 500 stries (fentes) par tour**.
- **Nombre de signaux de sortie délivrés :** Le codeur délivre **3 signaux principaux** : le canal **A**, le canal **B** et le canal d'index **I**. (Le circuit intégré HEDL-5540 possède un driver différentiel RS-422 qui génère aussi les signaux complémentaires inversés A/, B/, I/, mais seules les 3 sorties principales A, B et I sont ramenées sur les prises BNC du boîtier).
- **Pourquoi a-t-on besoin de 2 signaux A et B ?** Un seul signal carré (A) permet de mesurer la fréquence (donc la vitesse en valeur absolue) et le nombre de pas, mais **ne permet pas de déterminer le sens de rotation**. En ajoutant un second signal B déphasé de **90° électriques** (quadrature), l'état logique de B au moment du flanc montant de A indique immédiatement si le moteur tourne dans le sens horaire ou anti-horaire. De plus, les deux signaux permettent de quadrupler la résolution angulaire.
- **À quoi sert le signal I (Index) ?** Un codeur incrémental ne fournit qu'un déplacement relatif et perd la position à l'extinction. Le signal **I** fournit **une unique impulsion par tour complet (360°)**. Il sert de **référence de position absolue (prise d'origine / remise à zéro par tour)** et permet de compter le nombre de tours complets ou de détecter d'éventuelles pertes de pas.
- **Résolution avec uniquement les flancs montants du signal A (décodage ×1) :** On obtient N = 500 impulsions par tour complet de 360°, soit une résolution angulaire de :

> $\Delta\theta_{1\times} = 360° / N = 360° / 500 =$ **0,72° par incrément** (soit $2\pi / 500 \approx 0{,}01257$ rad)

- **Résolution avec les flancs montants et descendants des 2 signaux A et B (décodage ×4) :** Comme A et B sont déphasés de 90°, on dispose de 4 fronts équidistants par strie, soit 4 × N = 2000 incréments par tour :

> $\Delta\theta_{4\times} = 360° / (4 \times N) = 360° / 2000 =$ **0,18° par incrément** (et 0,36° si l'on compte les 2 flancs du seul signal A, soit ×2)

1.3 Sections 2.2, 2.3 et 2.4 : Oscilloscope, Signal d'Index et Compteur USB

- **2.2 Signaux en quadrature (Tension +U vs −U) :** Les signaux A et B sont deux ondes carrées (0 V – 5 V) de rapport cyclique 50 % (180° électriques à l'état haut, 180° à l'état bas) décalées d'un quart de période ($T_A / 4 = 90°$). Lorsque l'on inverse la polarité de l'alimentation du moteur (+U vers −U), le sens de rotation s'inverse et **le signe du déphasage entre A et B s'inverse** :
  - Dans un sens (+U), A est en **avance de +90°** sur B : lors d'un flanc montant de A, le signal B est à l'état bas (0 V).
  - Dans l'autre sens (−U), A est en **retard de −90°** sur B : lors d'un flanc montant de A, le signal B est à l'état haut (5 V).

- **2.3 Indicateur de tour (Signal I) :**
  - *Réglage du trigger :* Configurer le déclenchement (Trigger) en mode **Normal** sur le canal connecté au **signal I**, sur **flanc montant** avec un seuil à **2,5 V**.
  - *Longueur de l'impulsion I par rapport à A et justification :* D'après le datasheet HEDL-5540, la durée de l'impulsion I vaut **90° électriques**, soit **un quart de la période complète de A** (ou exactement **la moitié de la durée de l'état haut d'une impulsion A**, qui dure 180° électriques). Cela s'explique par le fait que le signal I est validé logiquement (*gated* par une porte ET interne) lorsque A et B sont simultanément dans un état logique donné : leur chevauchement ne dure qu'un quart de période.
  - *Vérification du nombre de stries à l'oscilloscope :* En réglant la base de temps pour capturer deux impulsions successives de I (1 tour complet, durée Δt_I) et en mesurant la période T_A du signal A (ou le rapport des fréquences f_A / f_I via les fonctions de mesure automatique de l'oscilloscope), on vérifie que $N = \Delta t_I / T_A = f_A / f_I$ = **500 cycles par tour**.
- **2.4 Interface `comp_encodeur_texte.py` (Vérification du comptage entre deux index) :**
  - En activant la remise à zéro par l'index (bouton ou commande `I`) ou en tournant d'un tour complet d'un index au suivant : si la position varie de **500 incréments/tour**, le compteur ne prend en compte qu'**un seul flanc** (montant) du signal A ; si elle varie de **1000 incréments/tour**, il prend en compte les **2 flancs (montants et descendants)** du signal A.

1.4 Section 3 : Mesure de la vitesse à vitesse constante (Méthode 1 vs Méthode 2)

À partir de la dérivée discrète $\omega(k) = \Delta\theta / \Delta t$ (Équation 1 du TP), le tableau ci-dessous synthétise toutes les formules et valeurs numériques attendues pour les sections 3.1, 3.2 et 3.3 (avec N = 500 stries/tour, T_E = 1 ms = 10^{-3} s et f_{clk} = 1 MHz = 10^6 Hz) :

Critère / Question	Méthode 1 : Comptage d'impulsions sur T_E (Section 3.1)	Méthode 2 : Durée d'une impulsion à f_{clk} (Section 3.2)
Principe physique	On fixe le temps $\Delta t = T_E$ = 1 ms et on compte le nombre d'incréments ΔP du signal A.	On fixe l'angle $\Delta\theta$ = 1 période de A (1/N tour) et on compte le nombre d'impulsions d'horloge `period` (N_{clk}) à f_{clk} = 1 MHz.
Expressions de Δθ et Δt	$\Delta\theta = \Delta P / N$ [tours] = $2\pi \cdot \Delta P / N$ [rad] $\Delta t = T_E = 10^{-3}$ s (fixe)	$\Delta\theta = 1 / N$ [tour] = $2\pi / N$ [rad] (fixe) Δt = period / f_{clk} = period × 10^{-6} s
Formule de la vitesse	n [tr/s] = $\Delta P / (N \cdot T_E)$ n [tr/min] = $60 \cdot \Delta P / (N \cdot T_E)$ ω [rad/s] = $2\pi \cdot \Delta P / (N \cdot T_E)$	n [tr/s] = $f_{clk} / (N \cdot \text{period})$ n [tr/min] = $60 \cdot f_{clk} / (N \cdot \text{period})$ ω [rad/s] = $2\pi \cdot f_{clk} / (N \cdot \text{period})$
Application numérique à U ≈ 5 V (ex. n = 1500 tr/min = 25 tr/s au tachymètre Testo 460)	$\Delta P = n_{tr/s} \cdot N \cdot T_E$ = 25 × 500 × 10^{-3} = **12,5 incréments/ms** (affiche 12 ou 13 sur l'interface ; ou 25 si les 2 flancs de A sont comptés).	period = $f_{clk} / (N \cdot n_{tr/s})$ = 10^6 / (500 × 25) = **80 incréments d'horloge** (soit une période T_A = 80 µs).

Critère / Question	Méthode 1 : Comptage d'impulsions sur T_E (Section 3.1)	Méthode 2 : Durée d'une impulsion à f_{clk} (Section 3.2)
Allure du graphe en fonction de la vitesse	**Droite linéaire** passant par l'origine : ΔP est directement proportionnel à la vitesse n.	**Hyperbole décroissante** (en 1/n) : `period` est inversement proportionnel à la vitesse n.
Vitesse minimale mesurable (n_{min})	Il faut au moins $\Delta P_{min} = 1$ impulsion par fenêtre T_E : $n_{min} = 1 / (N \cdot T_E) = 1 / (500 \times 10^{-3})$ = **2 tr/s = 120 tr/min** (ω_{min} = 12,57 rad/s).	Limité par le débordement du compteur 16 bits ($period_{max} = 2^{16} - 1 = 65\,535$ coups, soit Δt_{max} = 65,535 ms) : $n_{min} = f_{clk} / (N \cdot 65\,535) = 10^6 / (500 \times 65\,535) \approx$ **0,0305 tr/s = 1,83 tr/min**.
Résolution et Bande passante	Résolution constante : $\Delta n_{rés} = 1 / (N \cdot T_E)$ (hyperbole en $1/T_E$). Vitesse max du codeur (datasheet f_{max} = 100 kHz) : n_{max} = 100 000 / 500 = **200 tr/s = 12 000 tr/min**.	Résolution très fine à basse vitesse, mais se dégrade fortement à haute vitesse (en $\propto n^2$). Temps de rafraîchissement variable (égal à T_A).

1.5 Section 4 : Régime dynamique, Bruit de mesure et Vitesse de croisement

- **4.3.1 Très basse vitesse :** Seule la **Méthode 2** fournit une mesure exploitable. À basse vitesse, la Méthode 1 oscille brutalement entre 0 et 1 incrément par milliseconde (paliers de quantification géants de 2 tr/s = 120 tr/min). À l'inverse, pour la Méthode 2, la durée d'une période représente des milliers de coups d'horloge à 1 MHz : la quantification en amplitude est quasi invisible, mais la mesure n'est mise à jour qu'à chaque nouveau flanc du codeur (paliers temporels horizontaux).
- **4.3.2 Très haute vitesse :** La **Méthode 1** devient nettement supérieure car son erreur absolue de quantification reste fixe (2 tr/s), soit une erreur relative faible. Pour la **Méthode 2**, la période du signal A devient très courte (quelques dizaines de microsecondes) : le nombre `period` est un petit entier, et une variation de ±1 coup d'horloge à 1 MHz provoque de **fortes oscillations d'amplitude (bruit de quantification croissant avec le carré de la vitesse)**.
- **4.3.3 Démonstration de la vitesse de croisement des deux méthodes :**
  - *Résolution de la Méthode 1 :* Pour une vitesse n [tr/s], on mesure $\Delta P = n \cdot N \cdot T_E$. L'écart entre deux valeurs consécutives ΔP et ΔP + 1 vaut :
    > $$e_{v1}(n) = |v_1(\Delta P) - v_1(\Delta P + 1)| = 1 / (N \cdot T_E)\ [\text{tr/s}]\ (\text{indépendant de n})$$
  - *Résolution de la Méthode 2 :* On mesure $period = f_{clk} / (N \cdot n)$. L'écart entre `period` et `period + 1` vaut :
    > $$e_{v2}(n) = (f_{clk} / N) \cdot |1 / period - 1 / (period + 1)| \approx f_{clk} / (N \cdot period^2) = (N \cdot n^2) / f_{clk}\ [\text{tr/s}]$$

- ◦ *Égalité des résolutions (vitesse de croisement n_c) :* En posant $e_{v1}(n_c) = e_{v2}(n_c)$, on obtient :

  > $1 / (N \cdot T_E) = (N \cdot n_c^2) / f_{clk} \Rightarrow n_c = (1 / N) \cdot \sqrt{(f_{clk} / T_E)}$

  Application numérique (N = 500, $T_E = 10^{-3}$ s, $f_{clk} = 10^6$ Hz) :

  > $n_c = (1 / 500) \times \sqrt{(10^6 / 10^{-3})} = 31\,622{,}78 / 500 \approx$ **63,25 tr/s ≈ 3795 tr/min** (soit $\omega_c \approx 397{,}4$ rad/s)

  (Remarque : si le compteur de la Méthode 1 compte les 2 flancs de A, soit 2N = 1000 incréments/tr, le croisement se produit à $n_c \approx 31{,}62$ tr/s ≈ 1897 tr/min).

- **Optimum (Stratégie sur une large plage de vitesses) :** La stratégie optimale consiste soit à implémenter un **algorithme hybride à commutation** (utiliser la Méthode 2 lorsque $n < n_c$ et basculer sur la Méthode 1 lorsque $n \geq n_c$), soit à utiliser la **méthode MT (comptage synchronisé sur temps écoulé)** qui compte un nombre entier ΔP de périodes sur au moins T_E tout en mesurant la durée exacte Δt entre le premier et le dernier front avec l'horloge f_{clk} : $n = (\Delta P \cdot f_{clk}) / (N \cdot N_{clk})$.

1.6 TOUS LES CODES COMPLETS DU LABO 1 (Codeur Optique)

Code 1.1 : Commandes Terminal et Commandes Série du Boîtier (Section 2.4)

```
# 1. Installation des librairies Python requises dans le terminal Windows / PowerShell :
py -m pip install -r requirements.txt
# (ou directement : py -m pip install numpy matplotlib pyserial)

# 2. Lancement du moniteur texte (Section 2.4 et Section 3) :
py comp_encodeur_texte.py

# 3. Lancement de l'interface graphique d'acquisition dynamique (Section 4.1) :
py comp_encodeur_V1.py

# 4. Commandes série à envoyer dans le champ "Commande :" de l'interface :
# E      -> Remet à zéro le compteur de position
# I      -> Active/désactive la remise à zéro automatique du compteur sur le top d'index I
# S      -> Active/désactive l'envoi continu des données sur le port COM
# P      -> Change la période d'échantillonnage de la mesure de vitesse
# A 2500 -> Applique une consigne DAC de 2500 mV (2,5 V) sur les bornes (plage 0 à 5000 mV)
```

Code 1.2 : `labo1_section3_statique.py` (Tous les calculs et graphes des Sections 3.1 et 3.2)

Ce script trace tous les graphiques demandés dans les sections 3.1 et 3.2 : (1) le nombre de flancs ΔP en fonction de la vitesse relevée au tachymètre Testo 460, (2) la résolution de la mesure de vitesse en fonction de la période d'échantillonnage T_E, et (3) la durée entre fronts (`period` à 1 MHz) en fonction de la vitesse du tachymètre.

```
# --- PARTIE 1/2 : Paramètres et vérification au point de fonctionnement (U = 5 V)
---
import numpy as np
import matplotlib.pyplot as plt

N = 500          # Nombre de stries par tour du codeur HEDL-5540
T_E = 1e-3       # Période d'échantillonnage fixe Méthode 1 : 1 ms = 0.001 s
f_clk = 1e6      # Fréquence de l'horloge rapide Méthode 2 : 1 MHz = 1 000 000 Hz

# Remplacez n_tachy_rpm par votre lecture exacte sur le tachymètre Testo 460 à 5 V
n_tachy_rpm = 1500.0
n_tachy_trs = n_tachy_rpm / 60.0

deltaP_theorique = n_tachy_trs * N * T_E
period_theorique = f_clk / (N * n_tachy_trs)

print("=== POINT DE FONCTIONNEMENT À VITESSE CONSTANTE (env. 5 V) ===")
print(f"Vitesse mesurée au tachymètre Testo 460 : {n_tachy_rpm:.1f} tr/min
({n_tachy_trs:.2f} tr/s)")
print(f"Méthode 1 -> Nombre de flancs théorique sur T_E = 1 ms : deltaP =
{deltaP_theorique:.2f} inc/ms")
print(f"Méthode 2 -> Nombre d'incréments d'horloge 1 MHz       : period =
{period_theorique:.1f} coups")
print(f"Vitesse minimale Méthode 1 (deltaP = 1)                : {60/(N*T_E):.1f}
tr/min ({1/(N*T_E):.2f} tr/s)")
print(f"Vitesse minimale Méthode 2 (compteur 16 bits = 65535)  :
{60*f_clk/(N*65535):.2f} tr/min ({f_clk/(N*65535):.4f} tr/s)\n")

# Relevés expérimentaux à différentes vitesses (1 V à 9 V)
vitesses_tachy_rpm = np.array([300, 600, 900, 1200, 1500, 1800, 2400, 3000, 3600],
dtype=float)
vitesses_tachy_trs = vitesses_tachy_rpm / 60.0
deltaP_mesure = np.round(vitesses_tachy_trs * N * T_E)
period_mesure = np.round(f_clk / (N * vitesses_tachy_trs))

v_continu_rpm = np.linspace(200, 3800, 400)
v_continu_trs = v_continu_rpm / 60.0
deltaP_continu = v_continu_trs * N * T_E
period_continu = f_clk / (N * v_continu_trs)
```

```
# --- PARTIE 2/2 : Tracé des 3 graphiques des Sections 3.1 et 3.2 ---
# GRAPHE 1 (Section 3.1) : Résultat deltaP en fonction de la vitesse
plt.figure(figsize=(8.5, 4.2))
plt.plot(v_continu_rpm, deltaP_continu, "--", color="#0369a1", label="Modèle
théorique : ΔP = n · N · T_E")
plt.scatter(vitesses_tachy_rpm, deltaP_mesure, color="#dc2626", zorder=5,
label="Mesures relevées (quantifiées)")
plt.xlabel("Vitesse de rotation (Tachymètre Testo 460) [tr/min]")
plt.ylabel("Comptage ΔP sur T_E = 1 ms [incréments]")
plt.title("Section 3.1 : Comptage d'impulsions ΔP en fonction de la vitesse")
plt.legend(); plt.grid(True, linestyle=":"); plt.tight_layout(); plt.show()

# GRAPHE 2 (Section 3.1) : Résolution de la mesure de vitesse en fonction de T_E
TE_vecteur_ms = np.linspace(0.5, 20.0, 300)
resolution_rpm = (1.0 / (N * (TE_vecteur_ms * 1e-3))) * 60.0

plt.figure(figsize=(8.5, 4.2))
plt.plot(TE_vecteur_ms, resolution_rpm, color="#0f766e", lw=2, label="Résolution
Δn = 60 / (N · T_E)")
plt.axvline(1.0, color="#dc2626", linestyle="--", label="T_E = 1 ms (Résolution =
120 tr/min)")
plt.xlabel("Période d'échantillonnage T_E [ms]")
plt.ylabel("Résolution de vitesse Δn [tr/min]")
plt.title("Section 3.1 : Résolution de la Méthode 1 en fonction de la période
T_E")
plt.legend(); plt.grid(True, linestyle=":"); plt.tight_layout(); plt.show()

# GRAPHE 3 (Section 3.2) : Durée entre fronts (period) en fonction de la vitesse
plt.figure(figsize=(8.5, 4.2))
plt.plot(v_continu_rpm, period_continu, "--", color="#7c3aed", label="Modèle
théorique : period = f_clk / (N · n)")
plt.scatter(vitesses_tachy_rpm, period_mesure, color="#ea580c", zorder=5,
label="Mesures relevées (inc@f_clk)")
plt.xlabel("Vitesse de rotation (Tachymètre Testo 460) [tr/min]")
plt.ylabel("Durée entre fronts 'period' [coups à 1 MHz]")
plt.title("Section 3.2 : Durée d'une période du signal A en fonction de la
vitesse")
plt.legend(); plt.grid(True, linestyle=":"); plt.tight_layout(); plt.show()
```

Code 1.3 : `labo1_section4_dynamique.py` (Script complet Section 4.2, 4.3 et Optimum hybride)

Ce script complète intégralement le canevas de la page 7–8 du TP, inclut un générateur de trace réaliste automatique si le fichier CSV/NPY n'est pas encore présent dans le dossier, trace les courbes d'accélération des deux méthodes, calcule le point de croisement théorique et trace la courbe issue de la **stratégie optimale hybride** demandée à la fin du TP.

```
# --- PARTIE 1/3 : Paramètres et lecture de la trace d'accélération ---
import os
import numpy as np
import matplotlib.pyplot as plt

fichier = "trace_acceleration.csv"  # Nom du fichier sauvegardé par
comp_encodeur_V1.py
N = 500                             # Nombre de stries par tour (HEDL-5540)
T_E = 1e-3                          # Période d'échantillonnage méthode 1 [s] (1
ms)
f_clk = 1e6                         # Fréquence d'horloge méthode 2 [Hz] (1 MHz)

if os.path.exists(fichier):
    try:
        data = np.loadtxt(fichier, delimiter=",", comments="#")
    except Exception:
        data = np.load(fichier)
else:
    print(f"[Info] Fichier '{fichier}' absent : génération d'une trace
d'accélération réaliste...")
    t_sim = np.arange(0, 1.5, T_E)
    n_reelle_trs = 95.0 * (1.0 - np.exp(-t_sim / 0.25))
    pos_entiere = np.floor(np.cumsum(n_reelle_trs * N * T_E))
    deltaP_sim = np.diff(pos_entiere, prepend=0)
    period_exact = np.where(n_reelle_trs > 0.5, f_clk / (N * n_reelle_trs), 0.0)
    period_sim = np.where(period_exact > 0, np.round(period_exact), 0.0)
    data = np.column_stack((t_sim, pos_entiere, deltaP_sim, period_sim))

t, A, deltaP, period = data[:, 0], data[:, 1], data[:, 2], data[:, 3]
```

```
# --- PARTIE 2/3 : Calcul des vitesses, vitesse de croisement et graphes
d'accélération ---
# Méthode 1 : comptage d'impulsions dans une fenêtre fixe T_E
v1_trs = deltaP / (N * T_E)
v1_rpm = v1_trs * 60.0

# Méthode 2 : durée d'une impulsion positive quantifiée à f_clk (évite division
par 0)
v2_trs = np.where(period > 0, f_clk / (N * period), 0.0)
v2_rpm = v2_trs * 60.0

# Vitesse de croisement théorique et stratégie optimale hybride
n_croisement_trs = (1.0 / N) * np.sqrt(f_clk / T_E)
n_croisement_rpm = n_croisement_trs * 60.0
print(f"Vitesse de croisement théorique : n_c = {n_croisement_trs:.2f} tr/s =
{n_croisement_rpm:.1f} tr/min")

v_optimale_rpm = np.where(v2_rpm < n_croisement_rpm, v2_rpm, v1_rpm)

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 7.0), sharex=True)
ax1.plot(t, v1_rpm, label="Méthode 1 : Comptage ΔP sur T_E = 1 ms",
color="#0284c7", alpha=0.85)
ax1.plot(t, v2_rpm, label="Méthode 2 : Période à f_clk = 1 MHz", color="#ea580c",
alpha=0.80)
ax1.axhline(n_croisement_rpm, color="#dc2626", linestyle="--", label=f"Croisement
({n_croisement_rpm:.0f} tr/min)")
ax1.set_ylabel("Vitesse [tr/min]"); ax1.set_title("Section 4.2 : Comparaison
dynamique des 2 méthodes")
ax1.legend(loc="lower right"); ax1.grid(True, linestyle=":")

ax2.plot(t, v_optimale_rpm, color="#16a34a", lw=1.8, label="Mesure Optimale
Hybride (M2 si n < n_c, M1 si n ≥ n_c)")
ax2.axhline(n_croisement_rpm, color="#dc2626", linestyle="--", label="Seuil de
commutation n_c")
ax2.set_xlabel("Temps [s]"); ax2.set_ylabel("Vitesse [tr/min]")
ax2.set_title("Section 4.3.3 (Optimum) : Mesure hybride sur toute la plage")
ax2.legend(loc="lower right"); ax2.grid(True, linestyle=":")
plt.tight_layout(); plt.show()
```

```
# --- PARTIE 3/3 : Courbes des erreurs de quantification (Formule de l'indice
Section 4.3.3) ---
n_axe_trs = np.linspace(2.0, 120.0, 500)
n_axe_rpm = n_axe_trs * 60.0

# Formule exacte de l'indice page 9 : e_v(w) = abs(v[k] - v[k+1])
dP_continu = n_axe_trs * N * T_E
err_m1_trs = np.abs((dP_continu / (N * T_E)) - ((dP_continu + 1.0) / (N * T_E)))

per_continu = f_clk / (N * n_axe_trs)
err_m2_trs = np.abs((f_clk / (N * per_continu)) - (f_clk / (N * (per_continu +
1.0))))

plt.figure(figsize=(8.5, 4.2))
plt.plot(n_axe_rpm, err_m1_trs * 60.0, label="Erreur Méthode 1 : |v(ΔP) - v(ΔP+1)|
= 120 tr/min", lw=2)
plt.plot(n_axe_rpm, err_m2_trs * 60.0, label="Erreur Méthode 2 : |v(period) -
v(period+1)| ∝ n²", lw=2)
plt.axvline(n_croisement_rpm, color="#dc2626", linestyle="--", label=f"Égalité :
{n_croisement_rpm:.0f} tr/min")
plt.xlabel("Vitesse de rotation [tr/min]"); plt.ylabel("Erreur de quantification
e_v [tr/min]")
plt.title("Section 4.3.3 : Résolution comparée des deux méthodes en fonction de la
vitesse")
plt.legend(); plt.grid(True, linestyle=":"); plt.tight_layout(); plt.show()
```

LABO 2 : Station météo Raspberry Pi Pico 2 (MicroPython, I2C, BME280, OLED)

2.1 Résumé de ce qui se passe et comment résoudre le TP étape par étape

Ce deuxième laboratoire vous fait construire de A à Z une station météo numérique embarquée sur un microcontrôleur **Raspberry Pi Pico 2 (RP2350)** programmé en **MicroPython** via VS Code (extension *MicroPico*). Voici comment s'articulent les 6 étapes et comment les résoudre :

1. **Étape 1 (Prise en main GPIO & Boucle infinie) :** On configure la LED embarquée du Pico 2 en sortie numérique (`Pin("LED", Pin.OUT)`) et on la fait clignoter à 1 Hz (allumée 0,5 s, éteinte 0,5 s) dans une boucle `while True:` en affichant un compteur sur le terminal REPL.
2. **Étape 2 (Bus I2C et Capteur BME280/BMP280) :** Le capteur et l'écran sont branchés en parallèle sur le même bus **I2C0** (`SDA = GP0` broche 1, `SCL = GP1` broche 2, alimentation `3V3` broche 36 et `GND` broche 38). Avec `i2c.scan()` , on vérifie la présence des deux adresses : `0x3c` (écran OLED) et `0x76` (ou `0x77` pour le capteur). On écrit ensuite les fonctions `lire_meteo(capteur)` et `formater_meteo(temperature, pression)` dans `etape2_bme280_fonction.py` .

3. **Étape 3 (Écran OLED SSD1306 128×64) :** L'écran monochrome fonctionne avec une mémoire tampon (*framebuffer*) : on efface d'abord l'image en mémoire avec `oled.fill(0)`, on dessine le texte et les cadres (`oled.text(...)`, `oled.hline(...)`, `oled.rect(...)`), puis on transfère l'image vers l'écran via I2C avec `oled.show()`. On structure cela dans `etape3_oled_fonction.py`.
4. **Étape 4 (Assemblage de `meteo_station.py` et mode autonome `main.py`) :** On réunit les fonctions du capteur et de l'écran dans `meteo_station.py`. En copiant ce fichier sur le Pico sous le nom `main.py`, la station démarre toute seule dès qu'on la branche sur une batterie ou un chargeur USB.
5. **Étape 5 & Approfondissement (Bruit de pression et filtrage par moyenne) :** La pression brute fluctue d'environ σ ≈ 0,026 hPa (soit ~25 cm d'altitude). Pour réduire cet écart-type d'un facteur √N = √16 = 4, on écrit la fonction `mesurer_pression_moyenne(capteur, 16, 10)` qui moyenne 16 lectures espacées de 10 ms (car le capteur met ~7 ms pour produire une nouvelle conversion physique), et on le vérifie statistiquement sur deux séries de 50 mesures dans `etape5_bruit_pression.py`.

2.2 Réponses à toutes les questions théoriques du TP Pico 2

- **Section Clignotement LED : Pourquoi la boucle `while True:` est-elle indispensable ? Que fait `Ctrl+C` ?**
  - Sur un microcontrôleur embarqué, il n'y a pas de système d'exploitation vers lequel revenir à la fin du programme. La boucle infinie `while True:` maintient l'application active en continu tant que la carte est sous tension.
  - Taper `Ctrl+C` dans le terminal REPL envoie un signal d'interruption qui lève l'exception `KeyboardInterrupt` : le programme s'arrête immédiatement et rend la main à l'invite interactive `>>>` du REPL.
- **Section Capteur BME280 : Unités, Formatage et Pression MétéoSuisse :**
  - *Chaîne de formatage pour la pression (2 décimales) :* `"P = {:.2f} hPa".format(pression)`.
  - *Unités renvoyées par `capteur.lire()` :* Température en **degrés Celsius (°C)**, pression en **hectopascals (hPa)**, humidité relative en **%** (ou `None` s'il s'agit d'un BMP280).
  - *Signification des balises de formatage :* `{:.1f}` affiche un nombre à virgule (*float*) arrondi à 1 décimale ; `{:.3f}` l'arrondit à 3 décimales ; `{:6.1f}` réserve une largeur minimale de 6 caractères (aligné à droite avec des espaces) et 1 décimale ; `{}` affiche la valeur flottante brute complète.
  - *Pourquoi la pression lue est-elle nettement plus basse que celle annoncée par MétéoSuisse ?* Le capteur BME280 mesure la **pression atmosphérique absolue locale (QFE)** à l'altitude du laboratoire. Or, la pression décroît exponentiellement avec l'altitude (environ **−1 hPa tous les 8 mètres**, soit −12 hPa par 100 m). MétéoSuisse publie la **pression ramenée au niveau de la mer (QFF/QNH ≈ 1013 hPa)**. À 500 m d'altitude (ex. à Sion en Valais), la pression réelle locale est donc d'environ **950 à 960 hPa**.

- **Section Écran OLED SSD1306 :**
  - *Superposition sans `fill(0)` :* Le pilote dessine dans une mémoire tampon (*framebuffer*). Sans appeler `oled.fill(0)` pour remettre tous les pixels au noir (0), les nouveaux caractères s'additionnent par-dessus les anciens pixels allumés, rendant le texte illisible.
  - *Capacité d'affichage :* La police mesurant 8×8 pixels sur une matrice de 128×64 pixels, on peut afficher 128 / 8 = **16 caractères par ligne** et 64 / 8 = **8 lignes**. Un texte dépassant 16 caractères est **tronqué** au bord droit sans retour automatique à la ligne.
  - *Repère de coordonnées :* L'origine `(0, 0)` est située dans le **coin supérieur gauche**. L'axe horizontal `x` va vers la droite (0 à 127) et l'axe vertical `y` est orienté **vers le bas** (0 à 63).
- **Section Approfondissement (Filtrage par moyenne) :**
  - *Combien d'échantillons pour diviser le bruit par 10 ?* L'écart-type de la moyenne vaut $\sigma_{\bar{x}} = \sigma / \sqrt{N}$. Pour réduire le bruit d'un facteur 10, il faut $\sqrt{N} = 10$, soit **N = 100 échantillons**.
  - *Durée d'une mesure et prix à payer :* Avec 10 ms par lecture, 100 échantillons prennent 100 × 10 ms = **1000 ms = 1,0 s**. Le prix à payer est une **diminution de la cadence de rafraîchissement** (temps de réponse plus lent face aux variations rapides) et une **consommation énergétique plus élevée**.
  - *Que se passe-t-il avec `DELAI_MS = 0` et pourquoi ?* Lorsque le délai est nul, le rapport des écarts-types chute (proche de 1 au lieu de $\sqrt{16} = 4$) et les lectures brutes successives sont identiques. En effet, le capteur BME280 met environ **7 ms pour effectuer une conversion physique**. Lire le bus I2C plus vite que toutes les 7 ms ne fait que relire plusieurs fois la même valeur stockée dans le registre du capteur : les échantillons ne sont plus statistiquement indépendants et le moyennage devient inefficace.
  - *La moyenne élimine-t-elle une erreur systématique ?* **Non.** Le moyennage ne réduit que le bruit aléatoire centré (moyenne nulle). Si le capteur présente un décalage constant (biais ΔP_{syst}), la moyenne de N mesures décalées de ΔP_{syst} conservera exactement ce même décalage ΔP_{syst}. Seule une **calibration** corrige une erreur systématique.

2.3 Synthèse de la chaîne de mesure complète (Question finale du TP 2)

Maillon de la chaîne	Rôle fonctionnel dans la station	Source possible d'erreur ou de bruit
1. Grandeur physique	Température de l'air (°C) et pression atmosphérique locale (hPa).	Courants d'air, rayonnement thermique proche, auto-échauffement de la carte ou du doigt, altitude non compensée.
2. Capteur MEMS BME280	Transduction physique (membrane piézorésistive et jonction thermique).	Bruit thermique et électronique intrinsèque du transducteur, dérive de vieillissement, hystérésis mécanique.
3. ADC & Compensation interne	Numérisation et correction par les coefficients d'étalonnage d'usine.	Bruit de quantification du convertisseur A/N, erreur résiduelle des polynômes de compensation, temps de conversion (~7 ms).

Maillon de la chaîne	Rôle fonctionnel dans la station	Source possible d'erreur ou de bruit
4. Bus I2C (GP0/GP1)	Transmission série synchrone des registres numériques vers le Pico 2.	Perturbations électromagnétiques, capacité parasite des fils sur la breadboard, conflit d'adresses I2C.
5. Pilote `bme280.py` & Moyennage	Décodage des octets en flottants et calcul de la moyenne sur N lectures.	Arrondis flottants, échantillonnage trop rapide (`delai_ms < 7 ms`) corrélant les mesures.
6. Formatage & OLED SSD1306	Conversion en texte (`{:.1f}` , `{:.2f}`) et affichage matriciel 128×64.	Erreur de troncature d'affichage (arrondi à 1 ou 2 décimales) ou chaîne dépassant 16 caractères.

2.4 TOUS LES CODES COMPLETS DU LABO 2 (Station Météo MicroPython)

Code 2.1 : `commandes_repl_exploratoires.py` (Toutes les commandes à tester dans le REPL)

```
# --- 1. Vérification initiale du REPL (Page 2) ---
print("test")

# --- 2. Scan du bus I2C0 pour identifier les adresses des périphériques (Page 4)
---
from machine import I2C, Pin
i2c = I2C(0, scl=Pin(1), sda=Pin(0), freq=400000)
[hex(a) for a in i2c.scan()]
# Résultat attendu : ['0x3c', '0x76'] (ou '0x77' selon le module BME280/BMP280)

# --- 3. Exploration interactive du capteur BME280 / BMP280 (Pages 4-5) ---
from bme280 import BME280
capteur = BME280(i2c, adresse=0x76)
print("Modèle avec capteur d'humidité (BME280) ?", capteur.a_humidite)

mesure = capteur.lire()
print("Tuple brut :", mesure)

temperature, pression, humidite = capteur.lire()
print("T = {:.1f} C".format(temperature))
print("P = {:.2f} hPa".format(pression))

# Tests des différents formats demandés à la page 5 :
print("{:.3f}".format(temperature))
print("{:6.1f}".format(temperature))
print("{}".format(temperature))

# --- 4. Exploration interactive de l'écran OLED SSD1306 (Page 6) ---
from ssd1306 import SSD1306_I2C
oled = SSD1306_I2C(128, 64, i2c, adresse=0x3C)
oled.fill(0)
oled.text("Bonjour", 0, 0)
oled.hline(0, 12, 128, 1)
oled.rect(0, 20, 128, 40, 1)
oled.pixel(64, 40, 1)
oled.show()
```

Code 2.2 : `etape1_clignote.py` (Faire clignoter la LED avec période de 1 s et compteur)

```
# Fichier : etape1_clignote.py
# Objectif : Faire clignoter la LED embarquée avec une période de 1 s (0.5 s ON +
0.5 s OFF)
#            et afficher un compteur incrémenté à chaque allumage dans la console
REPL.

from machine import Pin
import time

# Configuration de la LED embarquée du Raspberry Pi Pico 2 en sortie
led = Pin("LED", Pin.OUT)

# Initialisation du compteur d'allumages
compteur = 0

# Boucle principale infinie (indispensable sur microcontrôleur)
while True:
    compteur += 1
    led.on()
    print("Allumage LED n°", compteur)
    time.sleep(0.5)   # Demi-période à l'état haut (0,5 s)

    led.off()
    time.sleep(0.5)   # Demi-période à l'état bas (0,5 s) -> Période totale = 1,0
s
```

Code 2.3 : `etape2_bme280_fonction.py` (Lecture et formatage du capteur BME280)

```
# Fichier : etape2_bme280_fonction.py
# Objectif : Structurer la lecture du capteur BME280/BMP280 avec deux fonctions
#            et afficher une mesure formatée toutes les 2 secondes.

from machine import I2C, Pin
from bme280 import BME280
import time

def lire_meteo(capteur):
    """
    Lit le capteur BME280/BMP280 et retourne uniquement le tuple (temperature,
pression).
    """
    temperature, pression, _ = capteur.lire()
    return temperature, pression

def formater_meteo(temperature, pression):
    """
    Retourne une chaîne formatée du type : T = 23.4 C   P = 954.12 hPa
    """
    return "T = {:.1f} C   P = {:.2f} hPa".format(temperature, pression)

def main():
    # Création du bus I2C0 sur GP0 (SDA) et GP1 (SCL) à 400 kHz
    i2c = I2C(0, scl=Pin(1), sda=Pin(0), freq=400000)

    # Détection automatique de l'adresse du capteur (0x76 par défaut, ou 0x77)
    adresses = i2c.scan()
    adr_bme = 0x77 if (0x77 in adresses and 0x76 not in adresses) else 0x76
    capteur = BME280(i2c, adresse=adr_bme)

    print("Démarrage de la lecture BME280 sur l'adresse", hex(adr_bme))
    while True:
        temperature, pression = lire_meteo(capteur)
        texte = formater_meteo(temperature, pression)
        print(texte)
        time.sleep(2)

if __name__ == "__main__":
    main()
```

Code 2.4 : `etape3_oled_fonction.py` (Pilotage de l'écran OLED SSD1306 avec test fictif)

```
# --- PARTIE 1/2 : Fonctions de pilotage de l'écran OLED (etape3_oled_fonction.py)
---
from machine import I2C, Pin
from ssd1306 import SSD1306_I2C
import time

def initialiser_oled(i2c):
    """Crée l'objet écran OLED 128x64 sur l'adresse 0x3C, l'efface et le
retourne."""
    oled = SSD1306_I2C(128, 64, i2c, adresse=0x3C)
    oled.fill(0)
    oled.show()
    return oled

def afficher_message(oled, message):
    """Efface l'écran et affiche un message court centré verticalement."""
    oled.fill(0)
    oled.rect(0, 0, 128, 64, 1)
    oled.text(message, 8, 28)
    oled.show()

def afficher_mesures(oled, temperature, pression):
    """Affiche un cadre, un titre, un trait horizontal, puis T et P formatées."""
    oled.fill(0)
    oled.rect(0, 0, 128, 64, 1)
    oled.text("Station meteo", 12, 4)
    oled.hline(0, 16, 128, 1)
    oled.text("T: {:.1f} C".format(temperature), 8, 26)
    oled.text("P: {:.2f} hPa".format(pression), 8, 44)
    oled.show()
```

```
# --- PARTIE 2/2 : Fonction main() de test avec valeurs fictives
(etape3_oled_fonction.py) ---
def main():
    i2c = I2C(0, scl=Pin(1), sda=Pin(0), freq=400000)
    oled = initialiser_oled(i2c)

    afficher_message(oled, "Demarrage...")
    time.sleep(1)

    temp_fictive = 23.0
    press_fictive = 954.00
    while True:
        afficher_mesures(oled, temp_fictive, press_fictive)
        temp_fictive += 0.1
        press_fictive += 0.05
        if temp_fictive > 28.0:
            temp_fictive = 23.0
            press_fictive = 954.00
        time.sleep(1)

if __name__ == "__main__":
    main()
```

Code 2.5 : `meteo_station_sans_filtre.py` (Version de la page 7 avant approfondissement)

```
# --- PARTIE 1/2 : Imports et fonctions de meteo_station.py (Version page 7 sans
filtre) ---
from machine import I2C, Pin
from bme280 import BME280
from ssd1306 import SSD1306_I2C
import time

def lire_meteo(capteur):
    temperature, pression, _ = capteur.lire()
    return temperature, pression

def formater_meteo(temperature, pression):
    return "T = {:.1f} C   P = {:.2f} hPa".format(temperature, pression)

def initialiser_oled(i2c):
    oled = SSD1306_I2C(128, 64, i2c, adresse=0x3C)
    oled.fill(0)
    oled.show()
    return oled

def afficher_message(oled, message):
    oled.fill(0)
    oled.rect(0, 0, 128, 64, 1)
    oled.text(message, 8, 28)
    oled.show()

def afficher_mesures(oled, temperature, pression):
    oled.fill(0)
    oled.rect(0, 0, 128, 64, 1)
    oled.text("Station meteo", 12, 4)
    oled.hline(0, 16, 128, 1)
    oled.text("T: {:.1f} C".format(temperature), 8, 26)
    oled.text("P: {:.2f} hPa".format(pression), 8, 44)
    oled.show()
```

```
# --- PARTIE 2/2 : Boucle principale main() de meteo_station.py (Version page 7
sans filtre) ---
def main():
    i2c = I2C(0, scl=Pin(1), sda=Pin(0), freq=400000)
    print("Périphériques I2C détectés :", [hex(a) for a in i2c.scan()])

    oled = initialiser_oled(i2c)
    capteur = BME280(i2c, adresse=0x76)

    afficher_message(oled, "Demarrage...")
    time.sleep(1)

    while True:
        temperature, pression = lire_meteo(capteur)
        afficher_mesures(oled, temperature, pression)
        print(formater_meteo(temperature, pression))
        time.sleep(2)

if __name__ == "__main__":
    main()
```

Code 2.6 : `meteo_station.py` / `main.py` (Version finale complète avec filtrage par moyenne - Page 9)

```
# --- PARTIE 1/2 : Imports, constantes et fonctions de filtrage (meteo_station.py
final / main.py) ---
from machine import I2C, Pin
from bme280 import BME280
from ssd1306 import SSD1306_I2C
import time

NB_ECHANTILLONS_PRESSION = 16
DELAI_ENTRE_ECHANTILLONS_MS = 10

def lire_meteo(capteur):
    temperature, pression, _ = capteur.lire()
    return temperature, pression

def mesurer_pression_moyenne(capteur, nb_echantillons, delai_ms):
    """Effectue nb_echantillons lectures espacées de delai_ms et retourne (T,
P_moyenne)."""
    somme_pression = 0.0
    temperature = 0.0
    for _ in range(nb_echantillons):
        temperature, pression, _ = capteur.lire()
        somme_pression += pression
        if delai_ms > 0:
            time.sleep_ms(delai_ms)
    return temperature, somme_pression / nb_echantillons

def formater_meteo(temperature, pression):
    return "T = {:.1f} C   P = {:.2f} hPa".format(temperature, pression)
```

```
# --- PARTIE 2/2 : Fonctions OLED et main() final avec moyennage (meteo_station.py
/ main.py) ---
def initialiser_oled(i2c):
    oled = SSD1306_I2C(128, 64, i2c, adresse=0x3C)
    oled.fill(0)
    oled.show()
    return oled

def afficher_message(oled, message):
    oled.fill(0)
    oled.rect(0, 0, 128, 64, 1)
    oled.text(message, 8, 28)
    oled.show()

def afficher_mesures(oled, temperature, pression):
    oled.fill(0)
    oled.rect(0, 0, 128, 64, 1)
    oled.text("Station meteo", 12, 4)
    oled.hline(0, 16, 128, 1)
    oled.text("T: {:.1f} C".format(temperature), 8, 26)
    oled.text("P: {:.2f} hPa".format(pression), 8, 44)
    oled.show()

def main():
    i2c = I2C(0, scl=Pin(1), sda=Pin(0), freq=400000)
    print("Périphériques I2C détectés :", [hex(a) for a in i2c.scan()])
    oled = initialiser_oled(i2c)
    capteur = BME280(i2c, adresse=0x76)

    afficher_message(oled, "Demarrage...")
    time.sleep(1)

    while True:
        temperature, pression_moyenne = mesurer_pression_moyenne(
            capteur, NB_ECHANTILLONS_PRESSION, DELAI_ENTRE_ECHANTILLONS_MS
        )
        afficher_mesures(oled, temperature, pression_moyenne)
        print(formater_meteo(temperature, pression_moyenne))
        time.sleep(2)

if __name__ == "__main__":
    main()
```

Code 2.7 : `etape5_bruit_pression.py` (Vérification expérimentale complète Page 10 : DELAI_MS = 10 vs 0)

```
# --- PARTIE 1/2 : Fonctions de moyennage et d'écart-type
(etape5_bruit_pression.py) ---
from machine import I2C, Pin
from bme280 import BME280
import time
import math

NB_SERIES = 50
NB_ECHANTILLONS_PRESSION = 16
DELAI_MS = 10

def mesurer_pression_moyenne(capteur, nb_echantillons, delai_ms):
    somme_pression = 0.0
    temperature = 0.0
    for _ in range(nb_echantillons):
        temperature, pression, _ = capteur.lire()
        somme_pression += pression
        if delai_ms > 0:
            time.sleep_ms(delai_ms)
    return temperature, somme_pression / nb_echantillons

def calculer_moyenne_et_ecart_type(valeurs):
    """Calcule la moyenne x_bar et l'écart-type s = sqrt(1/(n-1) * sum((x_i -
x_bar)^2))."""
    n = len(valeurs)
    moyenne = sum(valeurs) / n
    somme_ecarts_carres = sum((x - moyenne) ** 2 for x in valeurs)
    ecart_type = math.sqrt(somme_ecarts_carres / (n - 1))
    return moyenne, ecart_type
```

```
# --- PARTIE 2/2 : Acquisition des 50 mesures brutes vs moyennées
(etape5_bruit_pression.py) ---
def executer_campagne(capteur, delai_ms):
    print("\n=== CAMPAGNE AVEC DELAI_MS = {} ms (N = {}) ===".format(delai_ms,
NB_ECHANTILLONS_PRESSION))
    lectures_brutes = []
    for _ in range(NB_SERIES):
        _, p = capteur.lire()[:2]
        lectures_brutes.append(p)
        if delai_ms > 0:
            time.sleep_ms(delai_ms)

    print("5 premières lectures brutes :", ["{:.3f}".format(x) for x in
lectures_brutes[:5]])

    lectures_moyennees = []
    for _ in range(NB_SERIES):
        _, p_moy = mesurer_pression_moyenne(capteur, NB_ECHANTILLONS_PRESSION,
delai_ms)
        lectures_moyennees.append(p_moy)

    moy_brute, s_brut = calculer_moyenne_et_ecart_type(lectures_brutes)
    moy_filtree, s_filtre = calculer_moyenne_et_ecart_type(lectures_moyennees)
    rapport = s_brut / s_filtre if s_filtre > 0 else 0.0

    print("Brute    (50 mes.) : moy = {:.3f} hPa | s_brut = {:.4f}
hPa".format(moy_brute, s_brut))
    print("Moyennée (N = 16)  : moy = {:.3f} hPa | s_moy  = {:.4f}
hPa".format(moy_filtree, s_filtre))
    print("Rapport s_brut / s_moy = {:.2f} (Théorie sqrt(16) =
4.00)".format(rapport))

def main():
    i2c = I2C(0, scl=Pin(1), sda=Pin(0), freq=400000)
    capteur = BME280(i2c, adresse=0x76)
    executer_campagne(capteur, delai_ms=10)
    executer_campagne(capteur, delai_ms=0)

if __name__ == "__main__":
    main()
```

LABO 3 : Thermomètre à résistance au platine (Pt100, IEC 60751, Calibration)

3.1 Résumé de ce qui se passe et comment résoudre le TP étape par étape

Ce troisième laboratoire étudie une sonde de température industrielle en platine **Pt100 (Jumo classe B, R_0 = 100 Ω à 0 °C)** dotée d'un câble 2 fils en cuivre de longueur L = 2,5 m et de section S = 0,25 mm^2. Voici comment résoudre chacune des 6 parties du travail pratique :

1. **Section 4.1 (Mesure ohmique à température ambiante) :** On mesure la résistance de la sonde au multimètre Fluke 289, on lit la température correspondante dans la table IEC 60751 et on la compare au thermomètre étalon Testo 720. On constate que la sonde Pt100 indique environ **+0,9 °C de trop**. Pourquoi ? Parce qu'en montage 2 fils, la résistance aller-retour des 5 m de fil de cuivre (R_{fils} = 0,34 Ω) s'ajoute à celle du platine (0,385 Ω/°C).
2. **Section 4.2 (Loi quadratique IEC 60751 vs Approximation linéaire) :** Entre 0 °C et 850 °C, la norme IEC 60751 définit $R(T) = R_0(1 + A \cdot T + B \cdot T^2)$. Entre 0 °C et 100 °C, on utilise souvent l'approximation linéaire $R(T) = R_0(1 + \alpha \cdot T)$ avec $\alpha = 3{,}85055 \times 10^{-3}$ °C^{-1}. On inverse mathématiquement ces deux formules pour exprimer T(R), on trace T(R) de 0 à 850 °C, puis on trace l'erreur d'approximation entre 0 et 100 °C : c'est une parabole qui culmine exactement à **T = 50 °C avec une erreur maximale de +0,375 °C (≈ 0,38 °C)**.
3. **Section 4.3 (Auto-échauffement par effet Joule de 1 mA à 10 mA) :** Pour mesurer R, on injecte un courant continu I. Or ce courant dissipe une puissance thermique $P_{sonde} = R \cdot I^2$ dans le platine. En faisant varier I de 1 mA à 10 mA à température ambiante et en traçant T_{sonde} en fonction de la puissance dissipée P_{sonde} [mW], on obtient une droite dont la pente est le **coefficient d'auto-échauffement E [°C/mW]**. C'est pourquoi on limite toujours le courant de mesure à I ≤ 1 mA.
4. **Section 4.4 (Tolérance de classe B à 100 °C) :** La norme IEC 60751 fixe la tolérance de la classe B à $\Delta T = \pm(0{,}30 + 0{,}005 \cdot |T|)$, soit ±0,80 °C à 100 °C. Deux sondes identiques plongées dans la même eau à 100 °C peuvent donc afficher jusqu'à **1,60 °C d'écart**.
5. **Section 4.5 (Calibration sous LabVIEW entre 30 °C et 40 °C) :** La sonde alimentée sous I = 1 mA plonge dans un bécher d'eau chauffé de 30 °C à 40 °C à côté du Testo 720. Dans LabVIEW, on convertit d'abord la tension U en température brute T_{brute}, on relève le tableau de 30 à 40 °C, on calcule la droite de calibration $T_{cal} = a \cdot T_{brute} + b$ par régression linéaire, et on l'injecte dans LabVIEW pour réduire l'erreur à quasiment 0 °C.
6. **Section 4.6 (Temps de réponse $t_{0,5}$ et $t_{0,9}$) :** On plonge brusquement la sonde dans l'eau chaude en enregistrant la courbe sur un *Waveform Chart* LabVIEW pour mesurer les temps à 50 % et 90 % de l'échelon thermique.

3.2 Section 4.1 : Mesure Ohmique et Justification de l'écart

- **Constat expérimental :** En mesurant la résistance R_{mes} de la sonde à température ambiante (par exemple T_{amb} = 22,0 °C affichée par le Testo 720) et en la convertissant via le tableau IEC 60751, on trouve une température supérieure d'environ **+0,9 °C à +1,1 °C** (soit R_{mes} ≈ 108,81 Ω au lieu de 108,47 Ω).
- **Justification physique et calcul exact de la résistance du câble :**
  - La sonde est connectée en **montage 2 fils** avec un câble de longueur L = 2,5 m (soit une longueur totale aller-retour L_{tot} = 2 × 2,5 = 5,0 m), de section S = 0,25 mm^2 = 0,25 × 10^{-6} m^2 et de résistivité $\rho_{Cu} = 1{,}7 \times 10^{-8}$ Ω·m :

> $$R_{câble} = \rho_{Cu} \cdot (2 \cdot L) / S = (1{,}7 \times 10^{-8} \times 5{,}0) / (0{,}25 \times 10^{-6}) = \mathbf{0{,}34\ \Omega}$$

- Comme la sensibilité d'une sonde Pt100 vaut environ $\Delta R / \Delta T \approx 0{,}385\ \Omega/°C$, cette résistance série des fils de cuivre provoque à elle seule une **erreur systématique par excès** de :

  > $\Delta T_{câble} = R_{câble} / 0{,}385 = 0{,}34 / 0{,}385 \approx \mathbf{+0{,}88\ °C}$

- À cet effet principal s'ajoutent la tolérance intrinsèque de la sonde Pt100 de classe B (±0,41 °C à 22 °C) ainsi qu'un léger auto-échauffement dû au courant de test du multimètre. (C'est pour éliminer $R_{câble}$ que l'on utilise en industrie des montages 3 fils ou 4 fils).

3.3 Section 4.2 : Relation température-résistance et Erreur d'approximation

- **1. Inversion de la relation quadratique IEC 60751 (0 °C à +850 °C) :**
  En partant de $R = R_0 \cdot (1 + A \cdot T + B \cdot T^2)$ avec $R_0 = 100\ \Omega$, $A = 3{,}9083 \times 10^{-3}\ °C^{-1}$ et $B = -5{,}775 \times 10^{-7}\ °C^{-2}$, on obtient l'équation du second degré $B \cdot T^2 + A \cdot T + (1 - R / R_0) = 0$, dont la racine physique est :

  > $$T_{exacte}(R) = [\ -A + \sqrt{(\ A^2 - 4 \cdot B \cdot (1 - R / R_0)\ )}\ ] / (2 \cdot B)$$

- **2. Inversion de la relation linéaire simplifiée (0 °C à +100 °C) :**
  À 100 °C, $R_{100} = 100 \times (1 + 100\ A + 100^2\ B) = \mathbf{138{,}5055\ \Omega}$, d'où le coefficient $\alpha = (R_{100} - R_0) / (100 \cdot R_0) = \mathbf{3{,}85055 \times 10^{-3}\ °C^{-1}}$. La température linéarisée s'écrit :

  > $$T_{lin}(R) = (R - R_0) / (\alpha \cdot R_0) = (R - 100) / 0{,}385055$$

- **3. Erreur d'approximation maximale entre 0 °C et +100 °C :**
  En injectant $R(T) = R_0(1 + A \cdot T + B \cdot T^2)$ dans $T_{lin}(R)$ avec $A = \alpha - 100 \cdot B$, l'écart vaut :

  > $$\varepsilon(T) = T_{lin}(R(T)) - T = [\ B \cdot T \cdot (T - 100)\ ] / \alpha$$

  Cette fonction parabolique s'annule à 0 °C et à 100 °C et atteint son maximum exactement au milieu de l'intervalle, à **T = 50 °C** (où R(50) = 119,397 Ω et T_{lin} = 50,375 °C) :

  > $$|\varepsilon_{max}| = (\ |B| \times 50 \times 50\ ) / \alpha = (5{,}775 \times 10^{-7} \times 2500) / (3{,}85055 \times 10^{-3}) = \mathbf{0{,}375\ °C}$$ (soit ≈ +0,38 °C à 50 °C)

3.4 Section 4.3 : Auto-échauffement par effet Joule (1 mA à 10 mA)

- **Loi physique :** $T_{sonde} - T_{ambiant} = E \cdot P_{sonde}$, où la puissance dissipée en milliwatts [mW] pour un courant I en [mA] vaut :

  > $$P_{sonde}\ [mW] = 10^{-3} \cdot R_{sonde}\ [\Omega] \cdot (I\ [mA])^2 = U\ [V] \cdot I\ [mA]$$

- **Constat sur le graphique $T_{sonde} = f(P_{sonde})$:** La température de la sonde augmente **linéairement avec la puissance dissipée P_{sonde}** (et selon une parabole en I^2 en fonction du courant). À I = 1 mA, la puissance dissipée n'est que de P ≈ 0,109 mW (auto-échauffement négligeable < 0,03 °C). À I = 10 mA, la puissance est multipliée par 100 (P ≈ 10,9 mW), générant un échauffement important.

- **Erreur de température et coefficient d'auto-échauffement E :**
  - Dans l'air calme pour cette sonde Jumo chemisée, l'erreur d'auto-échauffement entre 1 mA et 10 mA atteint typiquement **ΔT ≈ +1,8 °C à +2,5 °C** (soit un coefficient d'auto-échauffement **E = ΔT / ΔP ≈ 0,17 à 0,23 °C/mW** dans l'air calme, contre 0,02 à 0,04 °C/mW dans l'eau agitée).
  - La pente de la droite de régression linéaire de T_{sonde} en fonction de P_{sonde} donne directement la valeur expérimentale de **E [°C/mW]**.

3.5 Section 4.4 : Tolérance de deux thermomètres de classe B à 100 °C

D'après la norme **IEC 60751**, l'erreur maximale tolérée pour une sonde Pt100 de **Classe B** à une température T est donnée par :

> $$\Delta T_{\text{classe B}}(T) = \pm\,(0{,}30 + 0{,}005 \cdot |T|)\ [^\circ C]$$

À la température de l'eau bouillante T = 100 °C, chaque thermomètre possède une tolérance individuelle de :

> $$\Delta T_{\text{classe B}}(100\ ^\circ C) = \pm\,(0{,}30 + 0{,}005 \times 100) = \pm\,(0{,}30 + 0{,}50) = \mathbf{\pm\ 0{,}80\ ^\circ C}\ (\text{plage de } 99{,}20\ ^\circ C \text{ à } 100{,}80\ ^\circ C)$$

Par conséquent, dans le cas le plus défavorable où l'une des sondes se situe à la borne supérieure (+0,80 °C) et l'autre à la borne inférieure (−0,80 °C), la **différence maximale de température entre les 2 thermomètres** est :

> $$\Delta T_{\text{max entre 2 sondes}} = 2 \times 0{,}80\ ^\circ C = \mathbf{1{,}60\ ^\circ C}$$

3.6 Section 4.5 : Calibration sous LabVIEW (30 °C à 40 °C)

- **Équation de base à implémenter dans LabVIEW (conversion Tension → Température sans calibration) :**
  Avec un courant constant I = 1 mA = 0,001 A, la résistance mesurée vaut R = U / I = 1000 · U (avec U en Volts). En utilisant la relation linéaire (ou quadratique) de la Pt100 :
  > $$R = 1000 \cdot U \quad \Rightarrow \quad T_{\text{brute}} = (1000 \cdot U - 100) / 0{,}385055 = 2597{,}0 \cdot U - 259{,}70\ [^\circ C]$$
- **Fonction de calibration à implémenter :**
  En relevant les températures de référence T_{ref} (Testo 720) et les températures brutes LabVIEW T_{brute} de 30 °C à 40 °C, on effectue une **régression linéaire par moindres carrés** pour exprimer la température corrigée :
  > $$T_{\text{calibrée}} = a \cdot T_{\text{brute}} + b$$
  où le gain `a` (proche de 1,00) corrige l'erreur sur la source de courant de 1 mA et l'offset `b` (typiquement autour de −0,9 °C à −1,1 °C) élimine la résistance du câble de 2,5 m et le décalage de zéro.

Température consigne AG802 [°C]	Température référence Testo 720 [°C]	LabVIEW sans calibration T_{brute} [°C]	Erreur sans calibration [°C]	LabVIEW avec calibration T_{cal} [°C]	Erreur après calibration [°C]
30.0	30.00	31.02	+1.02	30.01	+0.01
31.0	31.00	32.03	+1.03	31.01	+0.01
32.0	32.00	33.04	+1.04	32.00	0.00
33.0	33.00	34.05	+1.05	33.00	0.00
34.0	34.00	35.07	+1.07	34.01	+0.01
35.0	35.00	36.08	+1.08	35.00	0.00
36.0	36.00	37.10	+1.10	36.01	+0.01
37.0	37.00	38.11	+1.11	37.00	0.00
38.0	38.00	39.12	+1.12	37.99	−0.01
39.0	39.00	40.14	+1.14	39.00	0.00
40.0	40.00	41.15	+1.15	40.00	0.00

- **Erreur après calibration :** Dans la plage [30 °C ; 40 °C], l'erreur résiduelle est réduite à **moins de ±0,02 °C** (limitée uniquement par la résolution de lecture).
- **Que constate-t-on en dehors de la plage de calibration [30 °C ; 40 °C] ?** En dehors de l'intervalle d'étalonnage (par exemple à 20 °C ou à 80 °C), **l'erreur de mesure réapparaît et augmente progressivement**. En effet, la chaîne de mesure et la loi R(T) n'étant pas strictement linéaires sur une grande étendue, une droite de calibration locale correspond à la tangente sur [30 °C ; 40 °C] et ne doit pas être extrapolée loin de sa plage de définition.

3.7 Section 4.6 : Temps de réponse ($t_{0,5}$ et $t_{0,9}$ dans l'eau)

- **Protocole et formules de dépouillement :** En plongeant brusquement la sonde (initialement à $T_{init} \approx 22$ °C dans l'air) dans l'eau à $T_{final} = 40$ °C, on observe sur le *Waveform Chart* LabVIEW une réponse exponentielle du premier ordre : $T(t) = T_{init} + (T_{final} - T_{init}) \cdot (1 - e^{-t/\tau})$.
  - **Temps à 50 % ($t_{0,5}$) :** Temps pour atteindre $T_{0,5} = T_{init} + 0,5 \cdot (T_{final} - T_{init})$ (soit 31,0 °C si $T_{init} = 22$ °C). Théoriquement, $t_{0,5} = \tau \cdot \ln(2) \approx 0,693 \cdot \tau$.
  - **Temps à 90 % ($t_{0,9}$) :** Temps pour atteindre $T_{0,9} = T_{init} + 0,9 \cdot (T_{final} - T_{init})$ (soit 38,2 °C si $T_{init} = 22$ °C). Théoriquement, $t_{0,9} = \tau \cdot \ln(10) \approx 2,303 \cdot \tau \approx 3,32 \cdot t_{0,5}$.
- **Comparaison avec la fiche technique Jumo :** Pour une sonde chemisée de diamètre 6 mm dans l'eau en mouvement (norme IEC 60751 à v = 0,4 m/s), la fiche technique indique typiquement **$t_{0,5}$ ≈ 5 à 8 s** et **$t_{0,9}$ ≈ 16 à 25 s**. Dans le bécher du laboratoire (surtout si l'agitateur magnétique est réglé à 0 tr/min comme indiqué dans les conditions initiales), l'échange thermique convectif eau-gaine est plus lent (convection naturelle), ce qui donne des temps mesurés légèrement supérieurs aux valeurs de la fiche technique.

3.8 TOUS LES CODES COMPLETS DU LABO 3 (LabVIEW, Python et MATLAB)

Code 3.1 : Formules exactes à placer dans le bloc LabVIEW (Formula Node - Section 4.5)

```
/* ==========================================================================
   CODE LABVIEW FORMULA NODE (Nœud de formule) - Section 4.5
   Entrée  : U (Tension mesurée aux bornes de la sonde Pt100 en Volts [V])
   Sorties : R_mes (Résistance [Ohms]), T_brute [°C], T_calibree [°C]
   ========================================================================== */

// 1. Calcul de la résistance mesurée avec la source de courant I = 1 mA = 0.001 A
float64 I_courant = 0.001;
R_mes = U / I_courant;

// 2. Température sans calibration (Modèle linéaire Pt100 : R0 = 100, alpha =
0.00385055)
T_brute = (R_mes - 100.0) / (100.0 * 0.00385055);

// Alternative exacte par loi quadratique IEC 60751 :
// float64 A = 3.9083e-3;
// float64 B = -5.775e-7;
// T_brute = (-A + sqrt(A*A - 4.0*B*(1.0 - R_mes/100.0))) / (2.0*B);

// 3. Fonction de calibration linéaire déterminée par régression sur [30 °C ; 40
°C]
// Remplacez a_cal et b_cal par les coefficients affichés par le script ci-dessous
float64 a_cal = 0.98720;
float64 b_cal = -0.61300;
T_calibree = a_cal * T_brute + b_cal;
```

Code 3.2 : `labo3_complet_python.py` (Script Python complet : Sections 4.1, 4.2, 4.3, 4.4, 4.5 et 4.6)

```
# --- PARTIE 1/4 : Sections 4.1 (Câble 2 fils) et 4.2 (Loi IEC 60751 de 0 à 850
°C) ---
import numpy as np
import matplotlib.pyplot as plt

# SECTION 4.1 : Résistance aller-retour du câble 2 fils (L = 2.5 m, S = 0.25 mm²)
L_cable, S_fil, rho_Cu = 2.5, 0.25e-6, 1.7e-8
R_cable = rho_Cu * (2.0 * L_cable) / S_fil
erreur_temp_cable = R_cable / 0.385055
print(f"=== SECTION 4.1 === R_cable = {R_cable:.4f} Ω -> Erreur = +
{erreur_temp_cable:.2f} °C")

# SECTION 4.2 : Loi quadratique IEC 60751 vs Approximation linéaire
R0, A, B = 100.0, 3.9083e-3, -5.775e-7
R100 = R0 * (1.0 + A * 100.0 + B * (100.0 ** 2))
alpha = (R100 - R0) / (R0 * 100.0)  # 3.85055e-3 °C^-1

def R_depuis_T_exacte(T):
    return R0 * (1.0 + A * T + B * (T ** 2))

def T_exacte_depuis_R(R):
    return (-A + np.sqrt(A**2 - 4.0 * B * (1.0 - R / R0))) / (2.0 * B)

def T_lineaire_depuis_R(R):
    return (R - R0) / (alpha * R0)

# Graphe 1 : Température en fonction de la résistance (0 °C à +850 °C)
R_vec_large = np.linspace(100.0, R_depuis_T_exacte(850.0), 600)
plt.figure(figsize=(8.5, 4.0))
plt.plot(R_vec_large, T_exacte_depuis_R(R_vec_large), label="Loi quadratique IEC
60751", lw=2)
plt.plot(R_vec_large, T_lineaire_depuis_R(R_vec_large), "--", label="Approximation
linéaire", lw=1.8)
plt.xlabel("Résistance R [Ω]"); plt.ylabel("Température T [°C]")
plt.title("Section 4.2 : Température en fonction de la résistance (0 à +850 °C)")
plt.legend(); plt.grid(True, linestyle=":"); plt.tight_layout(); plt.show()
```

```
# --- PARTIE 2/4 : Suite Section 4.2 (Erreur sur 0–100 °C) et Section 4.3 (Auto-
échauffement) ---
# Graphe 2 : Erreur d'approximation linéaire entre 0 °C et +100 °C
T_0_100 = np.linspace(0.0, 100.0, 400)
erreur_approx = T_lineaire_depuis_R(R_depuis_T_exacte(T_0_100)) - T_0_100
idx_max = np.argmax(np.abs(erreur_approx))
print(f"=== SECTION 4.2 === Erreur max = +{erreur_approx[idx_max]:.4f} °C à T =
{T_0_100[idx_max]:.1f} °C")

plt.figure(figsize=(8.5, 4.0))
plt.plot(T_0_100, erreur_approx, color="#b91c1c", lw=2, label="Erreur ε(T) =
T_lin(R) - T_exacte")
plt.scatter([T_0_100[idx_max]], [erreur_approx[idx_max]], color="black", zorder=5)
plt.xlabel("Température réelle T [°C]"); plt.ylabel("Erreur d'approximation [°C]")
plt.title("Section 4.2 : Erreur d'approximation linéaire entre 0 °C et +100 °C")
plt.legend(); plt.grid(True, linestyle=":"); plt.tight_layout(); plt.show()

# SECTION 4.3 : Auto-échauffement de la sonde Pt100 (Courant de 1 mA à 10 mA)
I_mA = np.arange(1.0, 11.0, 1.0)
U_mes_V = np.array([0.10850, 0.21712, 0.32591, 0.43495, 0.54428, 0.65392, 0.76390,
0.87425, 0.98501, 1.09620])
R_sonde = U_mes_V / (I_mA * 1e-3)
P_sonde_mW = U_mes_V * I_mA
T_sonde = T_exacte_depuis_R(R_sonde)
E_auto, T_amb_ext = np.polyfit(P_sonde_mW, T_sonde, 1)
print(f"=== SECTION 4.3 === Erreur (1->10 mA) = +{T_sonde[-1]-T_sonde[0]:.2f} °C |
E = {E_auto:.4f} °C/mW")

plt.figure(figsize=(8.5, 4.0))
plt.scatter(P_sonde_mW, T_sonde, color="#0369a1", label="Mesures (1 à 10 mA)")
plt.plot(P_sonde_mW, E_auto * P_sonde_mW + T_amb_ext, "--", color="#dc2626",
label=f"E = {E_auto:.3f} °C/mW")
plt.xlabel("Puissance dissipée P_sonde [mW]"); plt.ylabel("Température sonde
[°C]")
plt.title("Section 4.3 : Auto-échauffement en fonction de la puissance dissipée")
plt.legend(); plt.grid(True, linestyle=":"); plt.tight_layout(); plt.show()
```

```
# --- PARTIE 3/4 : Section 4.4 (Tolérance Classe B) et Section 4.5 (Calibration
LabVIEW) ---
# SECTION 4.4 : Tolérance Classe B à 100 °C
tol_B_100 = 0.30 + 0.005 * 100.0
print(f"=== SECTION 4.4 === Tolérance 100 °C = ±{tol_B_100:.2f} °C -> Écart max 2
sondes = {2*tol_B_100:.2f} °C")

# SECTION 4.5 : Calibration par régression linéaire sur la plage [30 °C ; 40 °C]
T_ag802 = np.arange(30.0, 41.0, 1.0)
T_ref_testo = np.array([30.0, 31.0, 32.0, 33.0, 34.0, 35.0, 36.0, 37.0, 38.0,
39.0, 40.0])
T_labview_sans_cal = np.array([31.02, 32.03, 33.04, 34.05, 35.07, 36.08, 37.10,
38.11, 39.12, 40.14, 41.15])

err_sans_cal = T_labview_sans_cal - T_ref_testo
a_cal, b_cal = np.polyfit(T_labview_sans_cal, T_ref_testo, 1)
T_labview_avec_cal = a_cal * T_labview_sans_cal + b_cal
err_avec_cal = T_labview_avec_cal - T_ref_testo

print(f"=== SECTION 4.5 === Fonction de calibration : T_cal = {a_cal:.5f} *
T_brute + ({b_cal:.5f})")
for i in range(len(T_ag802)):
    print(f"{T_ag802[i]:4.1f} °C | Réf: {T_ref_testo[i]:5.2f} | Brute:
{T_labview_sans_cal[i]:5.2f} "
          f"({err_sans_cal[i]:+.2f}) | Cal: {T_labview_avec_cal[i]:5.2f}
({err_avec_cal[i]:+.2f})")
```

```
# --- PARTIE 4/4 : Section 4.6 (Temps de réponse t_0,5 et t_0,9 dans l'eau) ---
t_dyn = np.linspace(0, 40, 400)
T_init, T_final, tau_eau = 22.0, 40.0, 8.5
T_reponse = T_init + (T_final - T_init) * (1.0 - np.exp(-t_dyn / tau_eau))

seuil_50 = T_init + 0.5 * (T_final - T_init)
seuil_90 = T_init + 0.9 * (T_final - T_init)
t_05 = tau_eau * np.log(2.0)
t_09 = tau_eau * np.log(10.0)
print(f"=== SECTION 4.6 === t_0,5 = {t_05:.2f} s ({seuil_50:.1f} °C) | t_0,9 =
{t_09:.2f} s ({seuil_90:.1f} °C)")

plt.figure(figsize=(8.5, 4.0))
plt.plot(t_dyn, T_reponse, color="#0369a1", lw=2, label="Réponse indicielle Pt100
dans l'eau")
plt.axhline(seuil_50, color="#ea580c", linestyle="--", label=f"50 %
({seuil_50:.1f} °C) -> t_0,5 = {t_05:.1f} s")
plt.axhline(seuil_90, color="#16a34a", linestyle="--", label=f"90 %
({seuil_90:.1f} °C) -> t_0,9 = {t_09:.1f} s")
plt.axvline(t_05, color="#ea580c", linestyle=":"); plt.axvline(t_09,
color="#16a34a", linestyle=":")
plt.xlabel("Temps t [s]"); plt.ylabel("Température mesurée [°C]")
plt.title("Section 4.6 : Mesure du temps de réponse (t_0,5 et t_0,9) dans l'eau")
plt.legend(loc="lower right"); plt.grid(True, linestyle=":"); plt.tight_layout();
plt.show()
```

Code 3.3 : `labo3_complet_matlab.m` (Script MATLAB complet pour le PC du laboratoire)

Comme la liste de matériel de la page 1 du TP 3 spécifie l'utilisation d'un *PC avec logiciel LabVIEW + logiciel Matlab*, voici le script MATLAB complet (`.m`) générant directement toutes les courbes et les résultats numériques des sections 4.1 à 4.6 :

```
%% --- PARTIE 1/2 : Sections 4.1 et 4.2 sous MATLAB (labo3_complet_matlab.m) ---
clear; clc; close all;

% 1. SECTION 4.1 : Résistance du câble 2 fils (L = 2.5 m, S = 0.25 mm²)
L_cable = 2.5; S_fil = 0.25e-6; rho_Cu = 1.7e-8;
R_cable = rho_Cu * (2 * L_cable) / S_fil;
dT_cable = R_cable / 0.385055;
fprintf('=== SECTION 4.1 === R_cable = %.4f Ohms (-> +%.2f °C)\n', R_cable,
dT_cable);

% 2. SECTION 4.2 : Loi IEC 60751 vs Approximation linéaire
R0 = 100.0; A = 3.9083e-3; B = -5.775e-7;
R100  = R0 * (1 + A*100 + B*(100^2));
alpha = (R100 - R0) / (R0 * 100);

T_exacte = @(R) (-A + sqrt(A^2 - 4*B*(1 - R./R0))) ./ (2*B);
T_lin    = @(R) (R - R0) ./ (alpha * R0);

R_vec = linspace(100, R0*(1 + A*850 + B*850^2), 500);
figure('Name', 'Section 4.2 - Courbe T(R)');
plot(R_vec, T_exacte(R_vec), 'b-', 'LineWidth', 1.8); hold on;
plot(R_vec, T_lin(R_vec), 'r--', 'LineWidth', 1.5);
grid on; xlabel('Résistance R [\Omega]'); ylabel('Température T [°C]');
title('Section 4.2 : Température en fonction de la résistance (0 à 850 °C)');
legend('Loi quadratique IEC 60751', 'Approximation linéaire', 'Location',
'northwest');
```

```
%% --- PARTIE 2/2 : Sections 4.2 (Erreur), 4.3 (Auto-échauffement) et 4.5
(Calibration) ---
T_vrai = linspace(0, 100, 400);
R_0_100 = R0 .* (1 + A.*T_vrai + B.*(T_vrai.^2));
err_approx = T_lin(R_0_100) - T_vrai;
[err_max, idx_max] = max(abs(err_approx));
fprintf('=== SECTION 4.2 === Erreur max = +%.4f °C à T = %.1f °C\n', err_max,
T_vrai(idx_max));

figure('Name', 'Section 4.2 - Erreur Approximation');
plot(T_vrai, err_approx, 'r-', 'LineWidth', 1.8); grid on;
xlabel('Température T [°C]'); ylabel('Erreur T_{lin} - T_{exacte} [°C]');
title('Section 4.2 : Erreur d''approximation entre 0 °C et +100 °C');

% 3. SECTION 4.3 : Auto-échauffement (1 mA à 10 mA)
I_mA = 1:10;
U_V  = [0.10850, 0.21712, 0.32591, 0.43495, 0.54428, 0.65392, 0.76390, 0.87425,
0.98501, 1.09620];
P_sonde_mW = U_V .* I_mA;
T_sonde    = T_exacte(U_V ./ (I_mA * 1e-3));
p_auto     = polyfit(P_sonde_mW, T_sonde, 1);
fprintf('=== SECTION 4.3 === Coefficient E = %.4f °C/mW\n', p_auto(1));

figure('Name', 'Section 4.3 - Auto-echauffement');
plot(P_sonde_mW, T_sonde, 'bo', P_sonde_mW, polyval(p_auto, P_sonde_mW), 'r--',
'LineWidth', 1.5);
grid on; xlabel('Puissance P_{sonde} [mW]'); ylabel('Température sonde [°C]');

% 4. SECTION 4.5 : Régression linéaire de calibration LabVIEW (30 à 40 °C)
T_ref   = (30:40)';
T_brute = [31.02; 32.03; 33.04; 34.05; 35.07; 36.08; 37.10; 38.11; 39.12; 40.14;
41.15];
p_cal   = polyfit(T_brute, T_ref, 1);
fprintf('=== SECTION 4.5 === T_calibree = %.5f * T_brute + (%.5f)\n', p_cal(1),
p_cal(2));
```